UniVoucher

Security Review

March 13, 2026

Prepared by: Valves Security

Lead Auditors: Vesko210 | Merulez99

Contents

1 About Valves Security 3

2 Disclaimer 3

3 Risk Classification 3

4 Protocol Summary 4

5 Executive Summary 4

6 Findings 5

High Findings 5

[H-01] Partner address excluded from signed message enables front-running theft 5

Medium Findings 6

[M-01] Fee-on-transfer tokens cause undercollateralized slots 6

[M-02] Fixed-liability accounting is incompatible with rebasing and balance-mutating tokens 6

Low/Info Findings 7

[L-01] Dust deposit front-running blocks any `slotId` 7

[I-01] Cross-Deployment cardId Collision 7

1 About Valves Security

Valves Security is a Web3 security team founded by Veselin and Valeri. With deep experience across EVM-based protocols, Cosmos (Go) chains, and native Rust systems, we help projects identify and eliminate critical vulnerabilities. With a strong background and a strong team of freelancers we can asure a top quality audit.

Book a Security Review at valvessecurity.com or message us on X Valves Security

2 Disclaimer

The Valves team makes all effort to find as many vulnerabilities in the code in the given time period. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation. We stand by our process, yet encourage follow-up audits and meaningful bug bounty incentives for continued whitehat protection.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	**Critical**	**High**	**Medium**
Likelihood: Medium	**High**	**Medium**	**Low**
Likelihood: Low	**Medium**	**Low**	**Low**

Impact

- High - leads to a significant material loss of assets in the protocol or a group of users
- Medium - leads to a moderate material loss of assets in the protocol, moderately harms a group of users or to some disruption of the protocol's functionality
- Low - No funds at risk

Likelihood

- High - attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- Medium - only a conditionally incentivized attack vector, but still relatively likely
- Low - has unlikely assumptions or requires a significant stake by the attacker

4 Protocol Summary

UniVoucher introduces a decentralized crypto gift card primitive that allows any token to be distributed via simple voucher codes secured by on-chain smart contracts. By removing custodial control and enabling physical or digital distribution of crypto assets, the protocol bridges Web3 infrastructure with real-world gifting and reward systems.

5 Executive Summary

Overview

Project	UniVoucher
Commit Hash	5e2ec2752829c74680a00f9af847708c74b25a2b
Timeline	11/03/2026 – 13/03/2026

Scope

Id	Files in scope
1	src/*

Issues Found

Critical Risk	0
High Risk	1
Medium Risk	2
Low/Info Risk	2

6 Findings

High Findings

[H-01] Partner address excluded from signed message enables front-running theft

Description

The redemption signature covers only `cardId` and `to`, but not `partner`. Any caller can take a valid signature and submit it with an arbitrary `partner` address.

```
/**
 * @dev Redeem a gift card using its ID and signature.
 * @param cardId The ID of the gift card to redeem
 * @param to The address where to send the redeemed tokens
 * @param signature A signature proving ownership of the card secret
 * @param partner Optional partner address to receive 1% of the card amount (use address (0) for no partner)
 */
function redeemCard(string memory cardId, address payable to, bytes memory signature, address payable partner) external nonReentrant {
    // Code...

    // Calculate partner fee (1% if partner is provided, minimum 1 wei)
    uint256 partnerAmount = (partner != address(0)) ?
        (slotCopy.tokenAmount / 100 > 0 ? slotCopy.tokenAmount / 100 : 1) : 0;

    // Transfer the assets
    if (slotCopy.tokenAddress == address(0)) {
        // Transfer ETH
        if (partnerAmount > 0) {
            (bool sentPartner, ) = partner.call{value: partnerAmount}("");
            require(sentPartner, "Failed to send ETH to partner");
        }
        (bool sent, ) = to.call{value: slotCopy.tokenAmount - partnerAmount}("");
        require(sent, "Failed to send ETH");
    } else {
        // Transfer ERC20 tokens
        if (partnerAmount > 0) {
            IERC20(slotCopy.tokenAddress).safeTransfer(partner, partnerAmount);
        }
        IERC20(slotCopy.tokenAddress).safeTransfer(to, slotCopy.tokenAmount - partnerAmount);
    }

    emit CardRedeemed(cardId, slotId, to, slotCopy.tokenAddress, slotCopy.tokenAmount, partner, block.timestamp);
}
```

Impact

An MEV bot can front-run every redemption transaction, injecting itself as `partner` to steal 1% of the card value from every redemption on the protocol.

Proof of Concept

1. Alice broadcasts `redeemCard(cardId, to=Alice, sig, partner=address(0))`
2. Bob front-runs with `redeemCard(cardId, to=Alice, sig, partner=Bob)`
3. Bob receives `tokenAmount / 100`, Alice receives the remainder

Recommended Mitigation

Include `partner` in the signed message hash:

```
- bytes32 messageHash = keccak256(abi.encodePacked("Redeem card:", cardId, "to:", to));
+ bytes32 messageHash = keccak256(abi.encodePacked("Redeem card:", cardId, "to:", to, "
    partner:", partner));
```

Medium Findings

[M-01] Fee-on-transfer tokens cause undercollateralized slots

Description

`depositERC20` transfers `amount + fee` from the sender and records `amount` in the slot, but fee-on-transfer tokens deliver less than `amount + fee` to the contract.

Impact

The contract records more tokens than it holds. Redemptions and cancellations for the last slot will fail. This will lead to locking the last user funds.

Proof of Concept

1. Alice deposits 1000 DEFTOKEN (0.2% transfer fee) — contract records `tokenAmount = 1000`
2. Contract actually receives 998 tokens (consider protocol fee == 0)
3. On redemption of a given slot, `safeTransfer(to, 1000)` reverts — funds are stuck

Recommended Mitigation

Measure the actual received balance and use that as `tokenAmount`:

```
+ uint256 balanceBefore = IERC20(tokenAddress).balanceOf(address(this));
  IERC20(tokenAddress).safeTransferFrom(msg.sender, address(this), totalRequired);
+ uint256 actualAmount = IERC20(tokenAddress).balanceOf(address(this)) - balanceBefore -
    fee;
- _createCard(slotId, tokenAddress, amount, fee, message, encryptedPrivateKey);
+ _createCard(slotId, tokenAddress, actualAmount, fee, message, encryptedPrivateKey);
```

[M-02] Fixed-liability accounting is incompatible with rebasing and balance-mutating tokens

Description Each voucher stores a fixed tokenAmount at creation time, and that value is later used for redemption and cancellation. The protocol never reconciles stored liabilities against the contract's live balance for the token.

Impact If a token negatively rebases or otherwise reduces balances, stored voucher obligations can exceed the actual balance, causing redemption or cancellation failures.

Recommended Mitigation Explicitly disallow rebasing and balance-mutating tokens, or redesign the accounting model so obligations are derived from actual balances rather than immutable nominal amounts stored at deposit time.

Low/Info Findings

[L-01] Dust deposit front-running blocks any `slotId`

Description

`_createCard` rejects a `slotId` that is already registered. Anyone can register any `slotId`. An attacker can watch the mempool, front-run a victim's large deposit with a 1-wei deposit to the same `slotId`.

Impact The victim's transaction reverts. This can be repeated at near-zero cost to grief any user trying to create a card for somebody. Also this harms the protocol as the fee from the big deposit (that was not executed) is not recorded.

Proof of Concept

1. Alice broadcasts `depositETH(slotId=X, amount=10 ETH, ...)`
2. Bob front-runs with `depositETH(slotId=X, amount=1 wei, ...)`
3. Alice's transaction reverts with `"Slot ID already used"`

Recommended Mitigation

Require a minimum voucher value to make the attacker's griefing unsustainable.

[I-01] Cross-Deployment cardId Collision

Description `_generateCardId()` constructs a `cardId` by concatenating `versionPrefix`, `chainPrefix`, and `nextCardNumber`.

```
string memory cardId = string(abi.encodePacked(
    versionPrefix,
    chainPrefix,
    _uintToString(nextCardNumber)
));
```

Two contracts deployed with different but concatenation-ambiguous prefixes ("1"+"10" vs "10"+"1") produce identical IDs for the same counter value. Additionally, two contracts with identical prefixes deployed on different chains will produce fully identical `cardId` sequences.

Impact

The same `cardId` can exist on two destinct contracts where we have got a different `versionPrefix` and `chainPrefix`.

Recommendation Include `address(this)` in the `abi.encodePacked` call so each `cardId` is scoped to the specific contract instance, making it globally unique across all deployments:

```
function _generateCardId() internal returns (string memory) {
    string memory cardId = string(abi.encodePacked(
        versionPrefix,
        chainPrefix,
        _uintToString(uint256(uint160(address(this)))),
        _uintToString(nextCardNumber)
    ));
    nextCardNumber++;
    return cardId;
}
```